

R^4ec: A Reasoning, Reflection, and Refinement Framework for Recommendation Systems

Hao Gu*
Institute of Automation, Chinese Academy of Sciences
Beijing, China
guhao2022@ia.ac.cn

Rui Zhong*
Kuaishou Technology
Beijing, China
zhongrui@kuaishou.com

Yu Xia
University of Chinese Academy of Sciences
Beijing, China
xiayu24@mails.ucas.ac.cn

Wei Yang
Kuaishou Technology
Beijing, China
yangwei08@kuaishou.com

Chi Lu
Kuaishou Technology
Beijing, China
luchi@kuaishou.com

Peng Jiang
Kuaishou Technology
Beijing, China
jiangpeng@kuaishou.com

Kun Gai
Kuaishou Technology
Beijing, China
yuyue06@kuaishou.com

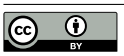
Abstract

Harnessing Large Language Models (LLMs) for recommendation systems has emerged as a prominent avenue, drawing substantial research interest. However, existing approaches primarily involve basic prompt techniques for knowledge acquisition, which resemble System-1 thinking. This makes these methods highly sensitive to errors in the reasoning path, where even a small mistake can lead to an incorrect inference. To this end, in this paper, we propose R^4ec, a reasoning, reflection and refinement framework that evolves the recommendation system into a weak System-2 model. Specifically, we introduce two models: an actor model that engages in reasoning, and a reflection model that judges these responses and provides valuable feedback. Then the actor model will refine its response based on the feedback, ultimately leading to improved responses. We employ an iterative reflection and refinement process, enabling LLMs to facilitate slow and deliberate System-2-like thinking. Ultimately, the final refined knowledge will be incorporated into a recommendation backbone for prediction. We conduct extensive experiments on Amazon-Book and MovieLens-1M datasets to demonstrate the superiority of R^4ec. We also deploy R^4ec on a large scale online advertising platform, showing 2.2% increase of revenue. Furthermore, we investigate the scaling properties of the actor model and reflection model. We also release the implementation code: https://github.com/ucas-hao/R4ec.git.

CCS Concepts

• **Information systems** → **Recommender systems**; **Retrieval models and ranking**.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.

RecSys '25, Prague, Czech Republic

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1364-4/25/09
https://doi.org/10.1145/3705328.3748068

Keywords

Large Language Model, Recommendation system, System-2 Thinking, Reflection and Refinement Mechanism

ACM Reference Format:
Hao Gu, Rui Zhong, Yu Xia, Wei Yang, Chi Lu, Peng Jiang, and Kun Gai. 2025. R^4ec: A Reasoning, Reflection, and Refinement Framework for Recommendation Systems. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25), September 22–26, 2025, Prague, Czech Republic.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3705328.3748068

1 Introduction

Nowadays, recommendation systems play a vital role in various online applications to alleviate the information overload problem and fulfill the information needs of users [3, 14, 43, 52, 57]. Besides, Large Language Models (LLMs) have achieved remarkable breakthroughs in Natural Language Processing (NLP), demonstrating impressive capacity in natural language understanding and text generation [1, 2, 34, 42]. Consequently, LLM-enhanced recommendation systems have received much attention and have been actively explored currently [4, 29, 50, 53].

At the core of integrating LLMs with recommendation systems is to harness LLM's extensive open-world knowledge and impressive reasoning capabilities to benefit recommendation systems. Initial attempts [11, 12, 44] has employed in-context learning to align LLMs with recommendation problems. They employ LLMs to rerank the candidate items filtered by traditional models (such as MF [25] and LightGCN [16]). However, these approaches fail to achieve satisfactory performance. Most recently, KAR [50] leverages chain-of-thought prompt technique, which facilitates the LLMs to break down recommendation tasks into a series of intermediate steps and generate the knowledge of LLMs regarding user preference and factual knowledge on items in a step-by-step manner. Then the extracted knowledge is treated as additional input features for downstream recommendation backbone. Although existing LLM-based

methods have achieved remarkable success, numerous challenges hinders their real-world applications.

On the one hand, they are sensitive to mistakes in the reasoning path [20, 49, 51], which means any mistake can lead to an incorrect answer. These shortcomings are attributed to the model's reliance on fast, intuitive System-1 thinking [22], which responds directly based on internally encoded perceptual information and world knowledge [20, 22]. To address this issue, inspired by the procedure of human cognition [17], We introduce an iterative reflection and refinement mechanism, facilitating slow and deliberate System-2 thinking. Specifically, we incorporate two models: an actor model and a reflection model. The actor model is capable of iteratively refining its responses based on feedback from the reflection model, shifting LLMs from System-1 thinking to a weak System-2 thinking.

On the other hand, these methods [50, 53] often necessitate numerous API calls, such as GPT-3.5 [1], leading to exorbitant inference latency and financial costs. This is typically intolerable in practical applications. Accordingly, we develop small LLMs like Qwen-2.5 7B [54] for user preference and item factual knowledge acquisition instead.

In this paper, we dive into how to employ System-2 thinking with small LLMs for recommendation tasks. To this end, we propose $\mathcal{R}^4$ec, a reasoning, reflection and refinement framework for enhancing recommendation system with LLMs. Specifically, we aim to train smaller LLMs to develop capabilities in reasoning, reflection, and refinement for user and item, respectively. As indicated in [51], the effectiveness of these mechanisms can be limited by factors such as model's capacity to accurately assess its own response. Thus, we employ a two-role paradigm, introducing two models: the actor model π_θ and the reflection model π_ψ. The actor model π_θ is tasked with reasoning about user preference or item factual knowledge and refining this knowledge based on feedback. In contrast, the reflection model π_ψ judges the rationality of the actor model's output and provides reflections for those outputs deemed unreasonable. The design philosophy of our approach is to enable the reflection and actor models to iteratively reflect and refine the knowledge until no further errors are detected, i.e., until the reflection model deems the knowledge to be rational. This process takes a step towards System-2 thinking, where deliberate and reflective reasoning is applied. Notably, we deploy distinct actor and reflection models for user preference and item factual knowledge respectively. Ultimately, the final refined user preference and item factual knowledge will be incorporated into the a recommendation backbone for prediction.

In summary, our main contributions are:

- We introduce $\mathcal{R}^4$ec framework, the first study within recommendation systems to explore System-2 thinking through iterative reflection and refinement mechanism, demonstrating its significant potential in the recommendation domain.
- We advance the reflection and refinement mechanisms in LLMs, transforming them from intuitive System-1 thinking to deliberate System-2 reasoning. We achieve this by incorporating an actor model that learns reasoning and refinement capabilities, and a reflection model that develops reflection abilities.
- We conduct comprehensive experiments on two public datasets and online industrial experiments to demonstrate the effectiveness of $\mathcal{R}^4$ec. Additionally, we shed light on the scaling properties of the actor and the reflection model.

2 Related Work

2.1 LLMs for Recommendation

Recently, leveraging Large Language Models (LLMs) for recommendation systems has attracted considerable attention [30, 31]. Generally, current LLM empowered recommenders can be primarily categorized into two main trends based on the distinct roles that LLMs play within the recommendation pipeline [53, 57]. (1) *LLM as a ranker*. This paradigm typically employs a frozen LLM to generate a ranked list that aligns with user interests [8, 44, 46]. However, the recommendation capabilities are somewhat limited due to the inherent gap between the LLMs' pre-training and recommendation tasks. Therefore, recent studies [4, 56] utilize instruction tuning for LLMs to inject recommendation capabilities. TallRec [4] fine-tunes LLAMA-7B [42] using LoRA [19], a parameter-efficient approach, in a two-stage tuning process. The first stage utilizes general data from Alpaca [41], followed by the second stage with recommendation data. (2) *LLM as a knowledge enhancer*. This method mainly utilizes LLMs to generate auxiliary information to improve the performance of recommendation models [53]. For example, [10] proposed a job recommendation model that leverages the summarization capabilities of LLMs to extract user job requirements. Similarly, KAR [50] leverages factual knowledge stored in LLMs to enhance movie and book recommendation.

2.2 Self-refine in LLMs

Self-refine, which aims to enhance the quality of responses from large language models (LLMs) by refining them using LLMs, have demonstrated effectiveness in various reasoning tasks [32, 49]. These tasks include arithmetic reasoning [26, 32, 48], open-domain question answering [8, 55], code generation [7, 21] and others.

The self-refine methods can be categorized into three groups based on the source of feedback. (1) *Intrinsic*. Intrinsic self-refine methods instruct LLMs to generate feedback on their own responses and correct them without external feedback. However, recent studies [33] report that intrinsic self-refine does not improve or even degrade performances. (2) *External Tools*. External tools employed for self-refine encompass code interpreters, which facilitate code generation tasks [7], and symbolic reasoners, applied to arithmetic reasoning [35]. Additionally, [13] builds a web search tool based on Google to retrieve information for validating correctness of the response. (3) *Fine-tuning*. This line of work improves reflection ability of open-source LLM by fine-tuning them on reflection datasets generated by GPT-4 [23, 24, 27]. For example, [36] first defines multiple error types for natural language reasoning tasks and develops specific feedback templates accordingly. Then they train a reflection generation model using synthetic feedback data. Our work falls under the third category, which involves fine-tuning reflection model to generate feedback given initial responses and fine-tuning refinement model to generate refined answers given the initial responses and feedback.

3 Method

3.1 Overview

Our proposed $\mathcal{R}^4$ec utilizes three primary capabilities, reasoning, reflection, and refinement, to develop System-2 thinking for recommendation. We formally define the key concepts:

- Reasoning capability refers to generating a response for a given question. In recommendation tasks, this involves inferring user preferences based on their interaction history for example.
- Reflection capability denotes the ability of LLMs to identify flaws in initial responses and to offer feedback for correction.
- Refinement capability involves generating a refined response based on initial responses and feedback.

We will introduce two small LLMs: an actor model π_θ for reasoning and refinement, and a reflection model π_ψ for reflection capability. Our primary motivation is to enable the actor model π_θ continuously refine its generated knowledge under the guidance of the reflection model π_ψ. This process facilitates the extraction of user preferences and item factual knowledge, gradually evolving into a more deliberate System-2 thinking approach. The final refined knowledge is subsequently integrated into a traditional recommendation backbone for prediction.

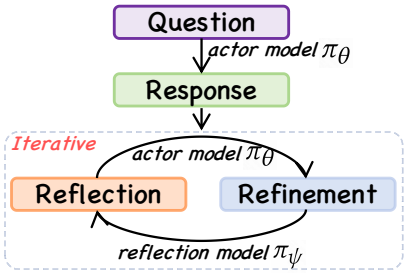


Figure 1: Our iterative reflection and refinement mechanism.

3.2 User Preference Dataset for $\mathcal{R}^4$ec

In a user preference inference task, given the interaction history and corresponding rating of a user, the model is required to summarize user preference. To construct reasoning, reflection and refinement dataset for user preference, we start from the widely available recommendation data $\{hist_k, item_k, label_k\}_{k=1}^{|D|}$, where $hist_k$, $item_k$ represent the interaction history of user u_k and the target item respectively. $label_k$ represents whether user u_k will like $item_k$, which corresponds to the gold answer.

We adopt a *reasoning while predicting* paradigm, which leverages the reasoning capabilities of LLMs to to provide predictions and justifications. Specifically, for a user u, given his interaction history $hist$ and the target item $item$, we prompt LLM $\mathcal{M}$ using the ***User Preference Reasoning Construction Prompt*** $\mathcal{P}^{user}_{reason}$ to predict whether the user u will like the target item $item$, along with rationales to justify the prediction. These rationales are natural language explanations that provide supporting reasoning for the prediction, which we set in the prompt as the user preference knowledge. Formally, this process can be formulated as:

$$< u_{pre}, pred > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{reason}(hist, item)) \quad (1)$$

where u_{pre} and $pred$ denotes user preference knowledge and the prediction indicating whether the user will like the item.

Upon failing to solve a problem, human students typically analyze the errors in their solutions and reflect on how to correct them. Inspired by this process, we design the reflection mechanism that enables the model to acquire the capability to identify flaws in the responses. Specifically, we prompt LLM $\mathcal{M}$ using the ***User Preference Reflection Construction Prompt*** $\mathcal{P}^{user}_{reflect}$. We task the LLM with judging whether the user preference knowledge u_{pre} is reasonable. For unreasonable user preference knowledge, $\mathcal{M}$ will generate reflections that not only pinpoint flaws in the responses but also provide valuable suggestions for correction. Formally, this process can be formulated as:

$$< judge^u, reflect^u > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{reflection}(hist, item, u_{pre})) \quad (2)$$

Here, $judge^u$ represents the assessment of whether the inferred user preference knowledge u_{pre} is reasonable, while $reflect^u$ refers to the reflection on u_{pre}. Notably, if LLM $\mathcal{M}$ deems u_{pre} to be reasonable, $reflect^u$ will be empty.

Given that we have access to the user's true preference for items, which we denote as $label$. Samples that lead to correct predictions and are deemed reasonable by $\mathcal{M}$ will be incorporated into the User Preference Reasoning Dataset $\mathcal{D}^u_{reason}$. Additionally, these samples will be included as positive instances in the User Preference Reflection Dataset $\mathcal{D}^u_{reflect}$. For user preference that lead to correct prediction but are deemed unreasonable by the $\mathcal{M}$, as well as those that lead to incorrect predictions but are considered reasonable, there is a high likelihood of issues in the reasoning or reflection process. Consequently, we discard these samples.

Finally, for samples that lead to incorrect predictions and are concurrently considered unreasonable by $\mathcal{M}$, we leverage these instances to develop a dataset specifically aimed at enhancing the model's refinement capabilities. To this end, we introduce the **User Preference Refine Construction Prompt** $\mathcal{P}^{user}_{refine}$, then engage $\mathcal{M}$ to generate a new prediction along with the refined user preference. This process can be represented as:

$$< u^r_{pre}, pred' > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{refine}(hist, item, u_{pre}, reflect^u)) \quad (3)$$

where u^r_{pre} denotes the refined user preference knowledge and $pred'$ represents the new prediction.

For samples where the new prediction $pred'$ matches the label, this indicates that not only the reflection successfully identifies errors in original user preference, but also the subsequent refinement yields the correct answer. We consider such reflections effective and will incorporate $s^u_{reflect} = \{(hist, u_{pre}), (judge^u, reflect^u)\}$ into the User Preference Reflection Dataset $\mathcal{D}^u_{reflect}$. Additionally, sample $\{(hist, u_{pre}, reflect^u), (u^r_{pre})\}$ will be added to the User Preference Refinement Dataset: $\mathcal{D}^u_{refine}$.

3.3 Item Factual Dataset for $\mathcal{R}^4$ec

To construct the dataset capable of endowing models with the abilities to reason, reflect, and refine upon item factual knowledge, we require supervision signals. However, these signals are not as intuitive as those found in the user preference data. To this end, we start from

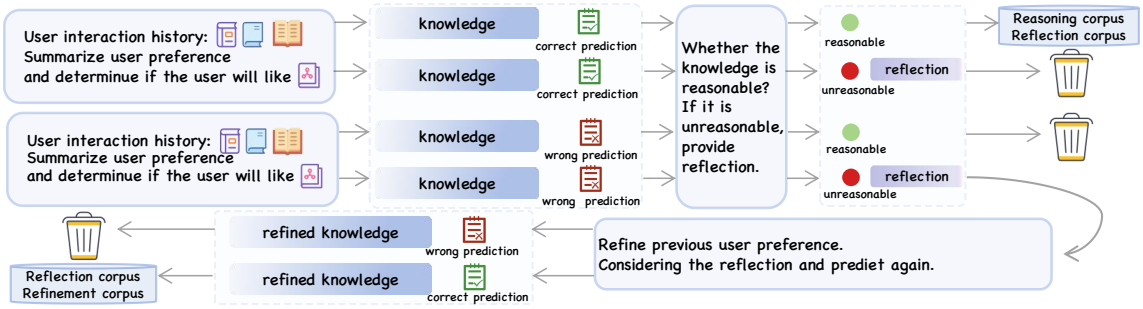


Figure 2: Overview of our user preference reasoning, reflection and refinement dataset construction process.

the following dataset structure $\{item_k, pos_k, neg_k, tar_k, label_k\}_{k=1}^{|D'|}$, where $item_k$ denotes the information of the target item i_k. pos_k and neg_k represent the interaction history of the users who like and dislike i_k, while tar_k denotes the interaction history of the target user. $label_k$ indicates whether the target user will like i_k, serving as the gold answer, i.e. supervision signal.

Building on the methodology used for constructing user preference datasets with capabilities for reasoning, reflection, and refinement, we apply a similar approach to develop the corresponding datasets for items. Suppose s^i_{reason}, $s^i_{reflect}$, s^i_{refine} denote sample from the item factual reasoning dataset $\mathcal{D}^i_{reason}$, item factual reflection dataset $\mathcal{D}^i_{reflect}$ and item factual refinement dataset $\mathcal{D}^i_{refine}$. Then, we can represent the obtained dataset as follows:

$$s^i_{reason} = \{(item, pos, neg), (i_{fact})\} \tag{4}$$

$$s^i_{reflect} = \{(item, pos, neg, i_{fact}), (judge^i, reflect^i)\} \tag{5}$$

$$s^i_{refine} = \{(item, pos, neg, i_{fact}, reflect^i), (i^r_{fact})\} \tag{6}$$

In this context, i_{fact}, $judge^i$, and $reflect^i$ correspond to the item factual knowledge, the assessment of whether this item factual knowledge is reasonable, and the reflection on this knowledge. Finally, i^r_{fact} is the refined item factual knowledge.

3.4 Training actor and reflection model

In Sec. 3.2 and Sec. 3.3, we describe how to construct the user preference and item factual datasets for $\mathcal{R}^4$ec, resulting in the datasets that enhance LLM's reasoning, reflection, and refinement capabilities for both users and items. In this section, we will explain how to train the actor and reflection models for users and items.

Without loss of generality, we denote $\mathcal{D}_{reason}, \mathcal{D}_{reflect}, \mathcal{D}_{refine}$ as the datasets that endow the models with reasoning, reflection, and refinement capabilities. Actor model and reflection model are denoted as π_θ and π_ψ.

First, we will train the actor model π_θ with basic reasoning and refinement ability. Refinement capability enables the generation of a revised response based on the problem, the previous response, and reflections on that response. In scenarios where both the previous response and its reflection are absent, this capability degenerates to basic reasoning. Recognizing this, we employ $\mathcal{D}_{reason}$ and $\mathcal{D}_{refine}$

Algorithm 1 User Preference Dataset Construction for $\mathcal{R}^4$ec

Input: Sample set $\mathcal{D} = \{(hist_k, item_k, label_k)_{k=1}^{|D|}\}$, LLM $\mathcal{M}$, reasoning construction prompt template $\mathcal{P}^{user}_{reason}$, reflection construction prompt template $\mathcal{P}^{user}_{reflect}$ and refinement construction prompt template $\mathcal{P}^{user}_{refine}$ for user preference.

Output: user preference reasoning dataset: $\mathcal{D}^u_{reason}$, user preference reflection dataset: $\mathcal{D}^u_{reflect}$ and user preference refinement dataset: $\mathcal{D}^u_{refine}$

Initialize $\mathcal{D}^u_{reason} = \mathcal{D}^u_{reflect} = \mathcal{D}^u_{refine} = \emptyset$
for each $(hist, item, label) \in \mathcal{D}$ **do**
  $< u_{pre}, pred > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{reason}(hist, item))$
  $< judge^u, reflect^u > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{reflect}(hist, item, u_{pre}))$
  if judge^u == "the user preference is reasonable" **and** pred = label **then**
    $\mathcal{D}^u_{reason}$ += $\{(hist), (u_{pre})\}$
    $\mathcal{D}^u_{reflect}$ += $\{(hist, u_{pre}), (judge^u, reflect^u = "")\}$
  else if judge^u == "the user preference is not reasonable" **and** pred ≠ label **then**
    $< u^r_{pre}, pred' > \leftarrow \mathcal{M}(\mathcal{P}^{user}_{refine}(hist, item, u_{pre}, reflect^u))$
    if $pred' = label$ **then**
      $\mathcal{D}^u_{reflect}$ += $\{(hist, u_{pre}), (judge^u, reflect^u)\}$
      $\mathcal{D}^u_{refine}$ += $\{(hist, u_{pre}, reflect^u), (u^r_{pre})\}$
    end if
  else
    $Discard(hist, item, label)$
  end if
end for

to equip actor model π_θ with both reasoning and refinement capabilities. The loss of training π_θ, i.e. $\mathcal{L}_{actor}$, is as follows:

$$\mathcal{L}_{reason} = \mathbb{E}_{(x,y)\sim\mathcal{D}_{reason}} [\log \pi_\theta(y|x)] \tag{7}$$

$$\mathcal{L}_{refine} = \mathbb{E}_{(x',y')\sim\mathcal{D}_{refine}} [\log \pi_\theta(y'|x')] \tag{8}$$

$$\mathcal{L}_{actor} = \mathcal{L}_{reason} + \mathcal{L}_{refine} \tag{9}$$

Here, (x, y) denotes sample from $\mathcal{D}_{reason}$ and (x', y') denotes sample from $\mathcal{D}_{refine}$. Loss functions $\mathcal{L}_{reason}$ and $\mathcal{L}_{refine}$ correspond to the reasoning and refinement capabilities, respectively.

Next, we equip the reflection model π_ψ with the reflection capability. Specifically, we train π_ψ through supervised fine-tuning with the collected reflection dataset $\mathcal{D}_{reflect}$. Suppose (x^*, y^*) represents sample from $\mathcal{D}_{reflect}$. Then the loss $\mathcal{L}_{reflect}$ for the reflection model π_ψ is as follows:

$$\mathcal{L}_{reflect} = \mathbb{E}_{(x^*,y^*)\sim\mathcal{D}_{reflect}} \left[\log \pi_\psi(y^*|x^*)\right] \tag{10}$$

In this way, we can obtain a reflection model that can provide constructive feedback on the reasoning paths of the actor model. Additionally, fine-tuning all parameters of the LLMs is time-consuming and resource-intensive [4, 29]. Thus, we employ the LoRA technique [19] to reduce computational demands while maintaining good performance.

3.5 Inference Strategy

To effectively utilize our learned reasoning, reflection, and refinement capabilities, we implement two distinct inference strategies: (1): Iterative Refinement and (2): Reflection as a Filter Function.[37, 39, 51, 58]. Next, we will demonstrate these strategies through user preference inference problem, while the strategy maintains equivalent applicability to item factual knowledge inference.

Iterative Refinement. This strategy is formalized as an iterative optimization process [38]. In the first trial, actor model π_θ will generate user preference knowledge. If the reflection model π_ψ identifies flaws in response, π_θ will incorporate the reflection from π_ψ to produce refined user preference knowledge. However, the refined user preference knowledge may still contain errors. Therefore, we can iteratively inspect the refined knowledge and refine it further if errors are identified. The final knowledge is only produced when it satisfies π_ψ (i.e., when π_ψ considers the user preference knowledge to be reasonable) or when the maximum number of retries is reached. The iterative reflection and refinement mechanism evolves our knowledge acquisition into a slow and deliberate System-2 thinking.

Reflection as a Filter. As indicated in previous studies [47, 51, 58], self-consistency technique is an effective approach for improving accuracy. Thus, we can generate several user preference knowledge via actor model π_θ, then the reflection model π_ψ is employed to filter out unreasonable user preference. The final user preference knowledge embedding is obtained by averaging the filtered preference. If no user preferences are deemed reasonable by π_ψ, we average all the available knowledge instead.

3.6 Knowledge Utilization

After we derive the final user preference knowledge u_{pre} and item factual knowledge i_{fact} from the actor and reflection model through the iterative refinement inference strategy. We need to transform the generated text-based knowledge into dense vectors. Specifically, we employ a knowledge encoder $\mathcal{E}ncoder$, such as BGE-M3 [6]:

$$e^u = \mathcal{E}ncoder(u_{pre}), e^i = \mathcal{E}ncoder(i_{fact}) \tag{11}$$

Table 1: Statistics of datasets used in this paper.

Dataset	Users	Items	Interactions
Amazon-Book	11906	17332	1.4 million
MovieLens-1M	6040	3706	1 million
Industrial Dataset	0.4 billion	10 million	2.3 billion

Here e^u and e^i denote the dense representations of user preference knowledge and item factual knowledge, respectively.

The recommendation task is generally formulated as a binary classification problem. Generally, we estimate the click probability as follows:

$$\hat{y} = \mathcal{M}(x, \mathcal{F}_u(e^u), \mathcal{F}_i(e^i)) \tag{12}$$

Here x represents categorical features for conventional recommendation systems. $\mathcal{M}$ represents the recommendation backbones, and $\mathcal{F}_u$ and $\mathcal{F}_i$ denote the connector for user preference knowledge and item factual knowledge, implemented as a MLP, respectively. During training, $\mathcal{F}_i$, $\mathcal{F}_u$ and $\mathcal{M}$ are jointly optimized via the binary cross-entropy loss.

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets. The datasets used in this paper are described as follows: **Amazon-Book** is the "Book" category of the Amazon Review Dataset. We regard reviews with ratings greater than 5 with positive. **MovieLens-1M** is a movie recommendation dataset with user-movie ratings ranging from 1 to 5. Samples with ratings greater than 3 are labeled as positive with the rest as negative. **Industrial Dataset** is collected from a large-scale advertising platform with hundreds of millions of users. Samples are constructed through sampling from impression logs. For academic datasets, we sort all interaction behaviors in chronological order and take the first 80% as the training set and the remaining as the testing set. Detailed statistics of the datasets are shown in Table 1.

4.1.2 Backbone Models. Because $\mathcal{R}^4$ec is a model-agnostic framework, various classic models can serve as the backbones. Here, we choose 6 representative CTR models. A brief introduction is provided below: **DIEN** [59] incorporates an interest evolution mechanism to capture the dynamic evolution of user interests over time. **GRU4Rec** [18] employs Gated Recurrent Units (GRU) combined with a ranking-based loss function to effectively model user sequences for recommendation systems. **AutoInt** [40] employs a self-attentive network enhanced with residual connections to model feature interactions. **FiGNN** [28] designs a novel model feature interaction graph network to utilize the strong representative of graphs. **DCN** [45] leverages a cross-network architecture to capture the bounded-degree feature interactions. **DeepFM** [15] adopts factorization machine to capture low-order and high-order feature interactions.

4.1.3 Evaluation Metrics. We utilize widely-used AUC (Area under the ROC curve) and LogLoss (binary cross-entropy loss) as evaluation metrics following previous studies [15, 40, 50]. **A higher AUC value or a lower Logloss value, even by a small margin (0.001**

User Preference Reasoning Construction Prompt

You are an expert recommender engine. Your task is to determine if the user will like the target book.
We will provide you with the following information:
- The user's historical interactions in the format [Book NAME, RATING], where a RATING greater than 4 indicates a like, and a RATING of 4 or less indicates a dislike.
- The target book's name.
<User History>{}</User History>
<Target Book Name>{}</Target Book Name>

Please strictly follow the following format:
<User Preference>[Summarize user preference on books in 200 words. Do not include any information about target book. You should think step by step.]</User Preference>
<Final Verdict>["user will like the target book" or "user will not like the target book" here]</Final Verdict>

User Preference Reflection Construction Prompt

Please act as an impartial judge. Below is a user's question and your previous inferenced user preference. Evaluate the user preference and provide valuable reflection.

<Question>We will provide you with the following information:
- The user's historical interactions in the format [Book NAME, RATING], where a RATING greater than 4 indicates a like, and a RATING of 4 or less indicates a dislike.
- The target book's name
<User History>{}</User History>
<Target Book Name>{}</Target Book Name>
Whether the user will like the target book?</Question>
<Previous User Preference>{}</Previous User Preference>

There may be potential issues with the user preference analysis. Your task is to judge the correctness of the user preference and provide valuable reflection to the user preference.
Please strictly follow the following format:

<Final Judgement>["The user preference is reasonable" or "The user preference is not reasonable"]</Final Judgement>
<Reflection>[If the user preference is not reasonable, you need provide variable reflection to the user preference. Identify key errors and potential misunderstanding here. (No more than 200 words). Do not include any information about the target book please. If the user preference is reasonable, nothing here]</Reflection>

User Preference Refine Construction Prompt

You are a helpful assistant for recommender system. Your task is to determine if the user will like the target book.

We will provide you with the following information:
- The user's historical interactions in the format [Book NAME, RATING], where a RATING greater than 4 indicates a like, and a RATING of 4 or less indicates a dislike.
- The target book's name.
- Your previous summarized user preference.
- Reflection on your previous summarized user preference.
<User History>{}</User History>
<Target Book Name>{}</Target Book Name>
<Previous user preference>{}</Previous user preference>
<Reflection>{}</Reflection>

You should refine the previous user preference considering the reflection and give the final correct answer.

Please strictly follow the following format:
<Refined User Preference> [Summarize user preference here in 200 words. Do not include any information about the target book please. You should think step by step.] </Refined User Preference>
<Final Verdict> ["user will like the target book" or "user will not like the target book" here] </Final Verdict>

Figure 3: Prompt template for constructing user preference reasoning, reflection and refinement dataset on Amazon-Book.

for example), can be viewed as a significant improvement, as indicated by prior research [50].

4.1.4 Baselines. We compare $\mathcal{R}^4$ec with the following settings: **Base**: we simply conduct experiments on conventional recommendation backbones that are introduced in Sec. 4.1.2 without knowledge from LLMs. **KAR**: Kar [50] acquires knowledge about users and items via the chain-of-thought technique, these knowledge will be employed as augment features for recommendation tasks. $\boldsymbol{\mathcal{R}^2}$**ec**: we employ only the obtained $\mathcal{D}_{reason}$ for training a single actor, i.e., without the reflection and refinement mechanisms.

4.1.5 Implementation Details. For dataset construction, we utilize API of a widely-used LLM *gpt-4o*. For the construction of the reasoning, reflection, and refinement dataset, we construct data from 40% of the users and items, and for each user or item, we generate one supervision sample. Additionally, we perform two separate inferences for each sample. We employ Qwen-2.5 7B [54] as our default actor model and reflection model. During training these LLMs, we set LoRA rank r as 8, LoRA alpha α as 16, and LoRA dropout as 0.05. The LoRA update matrices are applied on all linear layers. We fine-tune the LLMs for 3 epochs. During inference, we employ an iterative refinement strategy with the number of iterations set to 1 by default. Our implementations for conventional recommendation backbones follow [50]. For knowledge encoder, we use BGE-M3 [6] by default. Please refer our implementation code for more details, including prompt templates and so on.

4.2 Experimental Results

We implement $\mathcal{R}^4$ec, $\mathcal{R}^2$ec, and KAR [50] upon 6 representative CTR models. The experimental results are showcased in Table 2, which includes the AUC and its relative improvements, as well as the LogLoss and its relative reductions on both Amazon-Book and MovieLens-1M datasets. From these results, we can draw the following observations:

- Extracting user preference knowledge and item factual knowledge from LLMs significantly enhances the performance of recommendation systems. $\mathcal{R}^4$ec, $\mathcal{R}^2$ec, and KAR, designed to extract knowledge about users and items from LLMs, have facilitated the incorporation of this knowledge as sideinfo into downstream recommendation backbones. This integration has led to improvements in AUC and reductions in LogLoss across both the Amazon-Book and MovieLens-1M.
- Despite $\mathcal{R}^4$ec using the Qwen2.5-7b model and KAR employing GPT-3.5, $\mathcal{R}^4$ec achieves more pronounced improvements in AUC and reductions in LogLoss compared to KAR. Specifically, $\mathcal{R}^4$ec achieves a 1.36% greater relative improvement in AUC and a 3.29%

Backbones	Method	LLM	Amazon-Book				MovieLens-1M			
			AUC	Rel. Impr.	LogLoss	Rel. Impr.	AUC	Rel. Impr.	LogLoss	Rel. Impr.
DIEN [59]	Base	——	0.8280	——	0.5004	——	0.7755	——	0.5600	——
	KAR	GPT-3.5	0.8360	↑0.97%	0.4872	↓2.64%	0.7938	↑2.35%	0.5406	↓3.46%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8434	↑1.86%	0.4827	↓3.53%	0.7963	↑2.68%	0.5382	↓3.89%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8488	**↑2.51%**	0.4699	**↓6.09%**	0.8006	**↑3.23%**	0.5348	**↓4.50%**
GRU4Rec [18]	Base	——	0.8281	——	0.4992	——	0.7760	——	0.5589	——
	KAR	GPT-3.5	0.8376	↑1.15%	0.4915	↓1.54%	0.7942	↑2.34%	0.5401	↓3.36%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8410	↑1.56%	0.4825	↓3.35%	0.7955	↑2.51%	0.5407	↓3.25%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8492	**↑2.55%**	0.4690	**↓6.05%**	0.8002	**↑3.12%**	0.5370	**↓3.92%**
AutoInt [40]	Base	——	0.8261	——	0.5007	——	0.7736	——	0.5618	——
	KAR	GPT-3.5	0.8404	↑1.73%	0.4842	↓3.29%	0.7949	↑2.75%	0.5419	↓3.54%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8448	↑2.26%	0.4755	↓5.03%	0.7952	↑2.79%	0.5386	↓4.12%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8494	**↑2.82%**	0.4686	**↓6.41%**	0.8008	**↑3.52%**	0.5347	**↓4.82%**
FiGNN [28]	Base	——	0.8273	——	0.4993	——	0.7742	——	0.5611	——
	KAR	GPT-3.5	0.8393	↑1.45%	0.4826	↓3.34%	0.7947	↑2.65%	0.5422	↓3.37%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8452	↑2.16%	0.4752	↓4.83%	0.7968	↑2.92%	0.5374	↓4.22%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8495	**↑2.68%**	0.4712	**↓5.63%**	0.8021	**↑3.60%**	0.5344	**↓4.76%**
DCN [45]	Base	——	0.8271	——	0.4991	——	0.7746	——	0.5605	——
	KAR	GPT-3.5	0.8350	↑0.96%	0.4918	↓1.46%	0.7951	↑2.65%	0.5482	↓2.19%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8431	↑1.93%	0.4885	↓2.12%	0.7959	↑2.75%	0.5400	↓3.66%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8476	**↑2.48%**	0.4754	**↓4.75%**	0.8007	**↑3.37%**	0.5349	**↓4.57%**
DeepFM [15]	Base	——	0.8276	——	0.4986	——	0.7740	——	0.5616	——
	KAR	GPT-3.5	0.8370	↑1.14%	0.4858	↓2.56%	0.7953	↑2.73%	0.5397	↓3.89%
	$\mathcal{R}^2$ec	Qwen2.5-7B	0.8454	↑2.15%	0.4779	↓4.15%	0.7940	↑2.82%	0.5403	↓3.79%
	$\mathcal{R}^4$ec	Qwen2.5-7B	0.8483	**↑2.50%**	0.4704	**↓5.66%**	0.7998	**↑3.33%**	0.5366	**↓4.45%**

Table 2: Experimental results for different CTR backbones on Amazon-Book and MovieLens-1M datasets. We report AUC and Logloss. "Rel. Impr." is the relative improvement rate of method against each base model.

greater relative reduction in LogLoss than KAR. We speculate that the more pronounced improvements with $\mathcal{R}^4$ec compared to KAR stem from KAR's direct prompting of GPT-3.5 for knowledge acquisition, which might lead to hallucinations. In contrast, our $\mathcal{R}^4$ec can continuously identify and resolve issues, thereby significantly enhancing the reliability of the extraction of user preference knowledge and item factual knowledge.

- Comparing the $\mathcal{R}^4$ec with $\mathcal{R}^2$ec reveals that the user preference knowledge and item factual knowledge refined through our iterative reflection and refinement mechanism can lead to more significant improvements in downstream recommendation backbones. Specifically, on the Amazon-Book dataset, the $\mathcal{R}^4$ec outperformed $\mathcal{R}^2$ec, showing a 0.60% higher relative increase in AUC and a 1.93% greater relative reduction in LogLoss, averaged across six backbone models. Similarly, on the MovieLens-1M dataset, the $\mathcal{R}^4$ec demonstrated a 0.62% greater relative improvement in AUC and a 0.68% larger relative reduction in LogLoss compared to $\mathcal{R}^2$ec. These results validate the superiority of System-2 thinking over System-1 thinking, demonstrating significant potential in exploring System-2 thinking in recommendation systems.

4.3 Online Experimental

4.3.1 Experimental Setup. To validate the effectiveness of $\mathcal{R}^4$ec in real-world scenarios, we conduct online A/B experiments on a large-scale advertising platform. The traffic of the whole app is split into ten buckets uniformly. 20% of traffic is assigned to the online baseline while anther 10% is assigned to $\mathcal{R}^4$ec. As revealed in Tab. 1, our advertising platform serves over 400 million users, and the results collected from 10% of traffic for several weeks are convincing. We randomly sample one million samples for user and item reasoning, reflection, and refinement datasets construction, respectively. For the item factual knowledge extraction, we perform LLM inference across all items. In contrast, extracting user preference knowledge, which involves processing hundreds of millions of data points, results in high inference costs. Therefore, we tailor our inference strategies to the activity levels of users on the advertising platform. For active users, we conduct full inference using the LLM. For less active users, since the item knowledge has already been fully inferred, we approximate the inference results of the item knowledge in the user's historical behavior list as the inference results for their preference.

Method	Setting	Revenue	CVR
$\mathcal{R}^4$ec	all	↑2.2%	↑1.6%
	long-tail	↑4.1%	↑3.2%

Table 3: Results on online advertising platform.

4.3.2 Experimental Results. As indicated in Tab. 3, in a 14-day online A/B test, our method exhibits a 2.2% increase of revenue

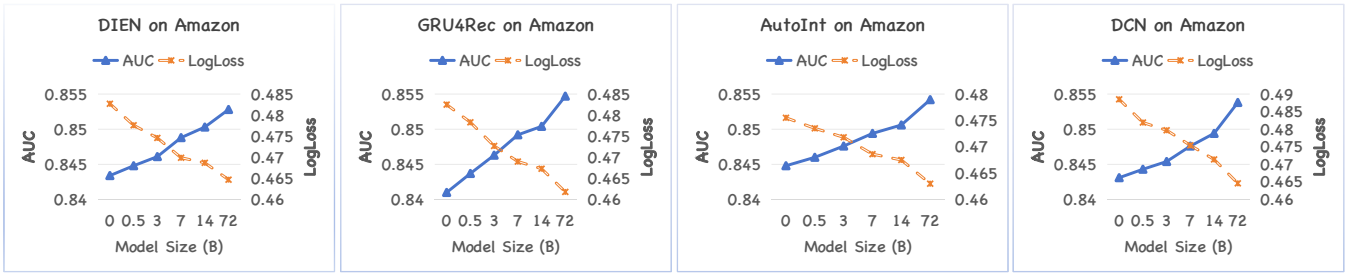


Figure 4: We use the Qwen-2.5 7B model as the actor and the Qwen-2.5 models with sizes 0.5B, 3B, 7B, 14B, and 72B as the reflection models. The AUC and LogLoss performance on the Amazon-Book dataset are shown.

and 1.6% improvement of the conversion rate compared with the baseline, resulting in significant business benefits. Furthermore, we utilize data with limited interactions to verify the effectiveness of cold starts. Experiment results in Tab. 3 demonstrate that $\mathcal{R}^4$ec can achieve a 4.1% increase of revenue and 3.2% improvement of conversion rate on long-tail data. This indicates that $\mathcal{R}^4$ec can be successfully implemented in industrial settings and improve recommendation experience for real-world users.

4.4 Ablation Study

Backbone	Encoder	Amazon-Book		MovieLens-1M	
		AUC	LogLoss	AUC	LogLoss
AutoInt	Base	0.8261	0.5007	0.7736	0.5618
	Bert	0.8449	0.4768	0.7934	0.5412
	Longformer	0.8462	0.4760	0.7953	0.5395
	BGE-M3	0.8494	0.4686	0.8008	0.5347

Table 4: Experimental results of different knowledge encoders on Amazon-Book and MovieLens-1M. We report AUC and LogLoss.

4.4.1 Effect of Different Knowledge Encoders. In this section, to investigate the impact of different knowledge encoders, we compare the AUC and LogLoss results of AutoInt on Amazon-Book and MovieLens-1M using BERT [9], Longformer [5], and BGE-M3 [6]as knowledge encoders. The experimental results are presented in Tab. 4.

From Tab. 4, we observe the following: (1) Regardless of the knowledge encoder used, both AUC significantly increases and LogLoss significantly decreases, demonstrating that extracting user preference and item factual knowledge can improve recommendation system performance. (2) BGE-M3 yields the most significant improvements, highlighting its superiority as knowledge encoder for recommendation system.

4.4.2 Scaling Properties of Reflection Model. To investigate the scaling law of the reflection model, namely, whether the performance improves as the size of the reflection model increases while the size of the actor model remains fixed. We conduct experiments on the Qwen-2.5 series models of varying sizes. The Qwen-2.5 7B is utilized as the actor model, paired with reflection models

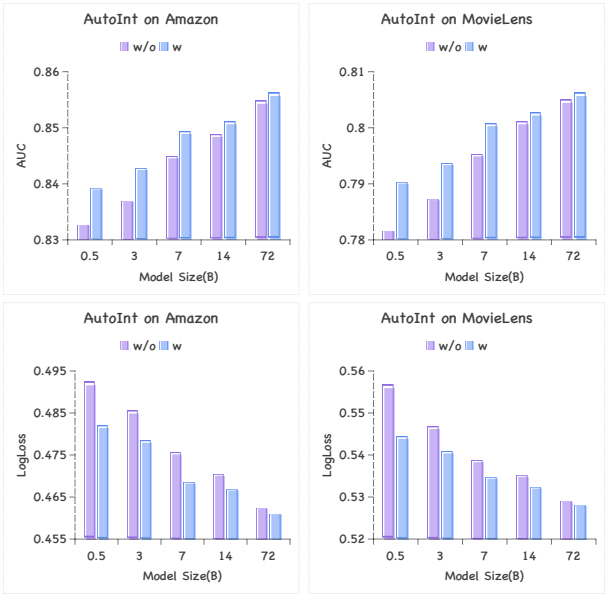


Figure 5: We use the Qwen-2.5 7B model as the reflection model and Qwen-2.5 models with sizes 0.5B, 3B, 7B, 14B, and 72B as the actor models.Here, 'w/o' and 'w/' denotes 'without a reflection model' and a 'with a reflection model', respectively.

of sizes 0.5B, 3B, 7B, 14B, and 72B. We employ DIEN, GRU4Rec, AutoInt, and DCN as recommendation backbones. The trends of AUC and LogLoss across different sizes of the reflection models on Amazon-Book are depicted in Fig. 4.

We find that as the size of the reflection model increases from 0.5B to 72B, there is a consistent improvement in AUC alongside a notable reduction in LogLoss. Specifically, as the size of the reflection model is increased from 0.5B to 72B, the average AUC on the Amazon dataset improves by a relative 1.3%, accompanied by a 3.8% relative reduction in LogLoss across four recommendation backbones. This trend underscores the scaling properties of reflection model, indicating that a larger reflection model can provide better feedback, thereby yielding knowledge that is more suitable for downstream recommendation models.

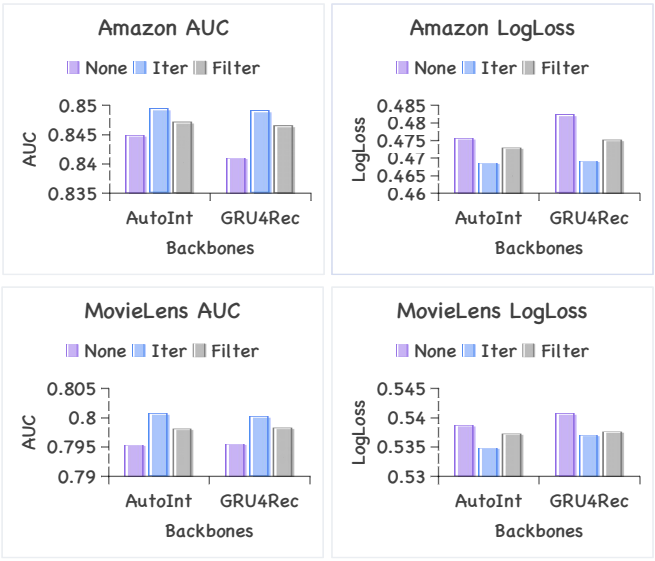


Figure 6: We compare the performance of AutoInt and GRU4Rec under the "Iterative Refinement" (Iter) and "Reflection as a Filter" (Filter) inference strategies.

4.4.3 Scaling Properties of Actor Model. In this section, we further explore the scaling properties of actor models. To this end, we use the Qwen-2.5 7B model [54] as the reflection model and Qwen-2.5 models with sizes 0.5B, 3B, 7B, 14B, and 72B as the actor models. Fig. 5 presents the AUC and LogLoss performance on Amazon-Book and MovieLens. We conduct experiments on AutoInt.

From Fig. 5, we observe the following: (1) Regardless of the actor model's scale, the inclusion of the 7B reflection model leads to consistent performance improvements. This result suggests that the 7B reflection model can consistently provide effective supervision, even when the actor model size reaches 72B. This indicates that even smaller reflection models can enhance larger actor models to a certain extent. (2) As the size of the actor model increases, the performance improvement from the reflection model tends to diminish.

4.4.4 Effect of Different Inference Strategies. In Sec. 3.5, we introduce two inference strategies "Iterative Refinement" and "Reflection as a Filter". For brevity, we refer to them as "Iter" and "Filter" respectively. In this section, we compare the performance of these two strategies. For "Filter" strategy, we perform three rounds of inference with the actor model, and the resulting knowledge is subsequently filtered through the reflection model. Fig. 6 presents the performance comparison of AutoInt and GRU4Rec on the Amazon and MovieLens datasets.

From Fig. 6, we can draw the following conclusions: (1) Both inference strategies lead to significant performance improvements, validating that the reflection mechanism helps acquire more reasonable and effective knowledge. (2) The "Iter" strategy consistently outperforms the "Filter" strategy, suggesting that using reflection alone is insufficient. Instead, it should be combined with refinement to correct unreasonable knowledge in recommendation system.

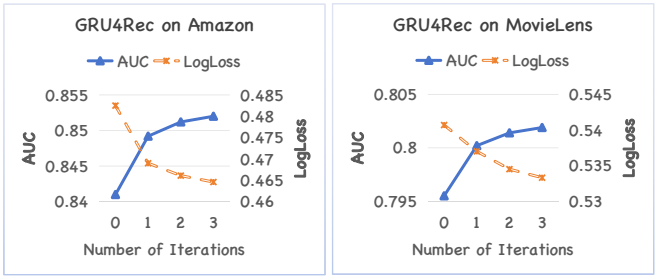


Figure 7: GRU4Rec's AUC and LogLoss performance on Amazon-Book and MovieLens with respect to the number of iterative refinement steps. We report AUC and LogLoss.

4.4.5 Effect of Iterative Refinement Steps. Since our iterative refinement inference strategy allows the reflection and actor models to continuously reflect and refine, the number of iterations becomes a critical hyperparameter. In this section, we investigate the impact of the number of iterative refinement steps on the performance of downstream recommendation models. Fig. 7 shows the performance of GRU4Rec on Amazon-Book and MovieLens-1M.

From Fig. 7, we observe that (1) by scaling up inference-time computation, i.e., increasing the number of iterative refinement steps, continuous improvements in AUC and reductions in LogLoss are achieved. This suggests that with each successive refinement, the acquired knowledge becomes more useful and rational. (2) As the number of iterations increases, the rate of improvement in AUC increases at a diminishing rate. We hypothesize that after multiple refinements, the reflection model's capacity becomes a bottleneck, , leading to the majority of the knowledge generated by the actor model being deemed reasonable by the reflection model. Finally, considering the scale of users and items in the recommendation system, as well as time and inference cost constraints, we adopt a single refinement step as the default method.

5 Conclusion

In this paper, we propose $\mathcal{R}^4$ec, a reasoning, reflection, and refinement framework that explores System-2 thinking within recommendation systems. Specifically, we introduce two models: the actor model, responsible for reasoning, and the reflection model, which evaluates the reasonableness of the actor's responses and provides feedback. This feedback encourages the actor model to refine its responses. Through this iterative process of reflection and refinement, we facilitate a slow and deliberate thinking process akin to System-2, ensuring more accurate acquisition of user preferences and factual item information. Ultimately, the backbone recommendation model integrates the refined knowledge from LLMs with original categorical features to improve recommendation performance. We demonstrate the effectiveness of $\mathcal{R}^4$ec through substantial improvements across Amazon-Book and MovieLens-1M. Additionally, our results validate the scaling properties of the actor model and reflection model. We hope that our work will inspire further research into advancing System-2 thinking in recommendation systems.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[3] Zhuoxi Bai, Ning Wu, Fengyu Cai, Xinyi Zhu, and Yun Xiong. 2024. Finetuning Large Language Model for Personalized Ranking. *arXiv preprint arXiv:2405.16127* (2024).

[4] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1007–1014.

[5] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).

[6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216* (2024).

[7] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).

[8] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering chatgpt's capabilities in recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1126–1132.

[9] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[10] Yingpeng Du, Di Luo, Rui Yan, Xiaopei Wang, Hongzhi Liu, Hengshu Zhu, Yang Song, and Jie Zhang. 2024. Enhancing job recommendation through llm-based generative adversarial networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8363–8371.

[11] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. 2023. Chat-rec: Towards interactive and explainable llms-augmented recommender system. *arXiv preprint arXiv:2303.14524* (2023).

[12] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*. 299–315.

[13] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2023. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738* (2023).

[14] Hao Gu, Jiangyan Yi, Chenglong Wang, Jianhua Tao, Zheng Lian, Jiayi He, Yong Ren, Yujie Chen, and Zhengqi Wen. 2025. *ALLM4ADD*: Unlocking the Capabilities of Audio Large Language Models for Audio Deepfake Detection. *arXiv preprint arXiv:2505.11079* (2025).

[15] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).

[16] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.

[17] Georg Wilhelm Friedrich Hegel. 1991. *The Encyclopaedia Logic, with the Zus tze: Part I of the Encyclopaedia of Philosophical Sciences with the Zusätze*. Vol. 1. Hackett Publishing.

[18] B Hidasi. 2015. Session-based Recommendations with Recurrent Neural Networks. *arXiv preprint arXiv:1511.06939* (2015).

[19] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[20] Yixin Ji, Juntao Li, Hai Ye, Kaixin Wu, Jia Xu, Linjian Mo, and Min Zhang. 2025. Test-time Computing: from System-1 Thinking to System-2 Thinking. *arXiv preprint arXiv:2501.02497* (2025).

[21] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Soneya Binta Hossain, Baishakhi Ray, Varun Kumar, Xiaofei Ma, and Anoop Deoras. 2024. Training LLMs to Better Self-Debug and Explain Code. *arXiv preprint arXiv:2405.18649* (2024).

[22] Daniel Kahneman. 2011. Thinking, fast and slow. *Farrar, Straus and Giroux* (2011).

[23] Pei Ke, Bosi Wen, Zhuoer Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, et al. 2023. Critiquellm: Scaling llm-as-critic for effective and explainable evaluation of large language model generation. *arXiv preprint arXiv:2311.18702* (2023).

[24] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. 2024. Language models can solve computer tasks. *Advances in Neural Information Processing Systems* 36 (2024).

[25] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[26] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, et al. 2024. Training language models to self-correct via reinforcement learning. *arXiv preprint arXiv:2409.12917* (2024).

[27] Junlong Li, Shichao Sun, Weizhe Yuan, Run-Ze Fan, Hai Zhao, and Pengfei Liu. 2023. Generative judge for evaluating alignment. *arXiv preprint arXiv:2310.05470* (2023).

[28] Zekun Li, Zeyu Cui, Shu Wu, Xiaoyu Zhang, and Liang Wang. 2019. Fi-gnn: Modeling feature interactions via graph neural networks for ctr prediction. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 539–548.

[29] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2023. Llara: Aligning large language models with sequential recommenders. *arXiv preprint arXiv:2312.02445* (2023).

[30] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Bridging items and language: A transition paradigm for large language model-based recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1816–1826.

[31] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. *arXiv preprint arXiv:2304.10149* (2023).

[32] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2024).

[33] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2023. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations*.

[34] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[35] Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. *arXiv preprint arXiv:2305.12295* (2023).

[36] Debjit Paul, Mete Ismayilzada, Maxime Peyrard, Beatriz Borges, Antoine Bosselut, Robert West, and Boi Faltings. 2023. Refiner: Reasoning feedback on intermediate representations. *arXiv preprint arXiv:2304.01904* (2023).

[37] Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. 2024. Recursive introspection: Teaching language model agents how to self-improve. *arXiv preprint arXiv:2407.18219* (2024).

[38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).

[39] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).

[40] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. Autoint: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1161–1170.

[41] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: an instruction-following llama model (2023). *URL https://github. com/tatsu-lab/stanford_alpaca* 1, 9 (2023).

[42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[43] Hanbing Wang, Xiaorui Liu, Wenqi Fan, Xiangyu Zhao, Venkataramana Kini, Devendra Yadav, Fei Wang, Zhen Wen, Jiliang Tang, and Hui Liu. 2024. Rethinking large language model architectures for sequential recommendations. *arXiv preprint arXiv:2402.09543* (2024).

[44] Lei Wang and Ee-Peng Lim. 2023. Zero-shot next-item recommendation using large pretrained language models. *arXiv preprint arXiv:2304.03153* (2023).

[45] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.

[46] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Tokenizer for LLM-based Generative Recommendation. *arXiv preprint arXiv:2405.07314* (2024).

[47] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[48] Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. 2022. Generating sequences by learning to self-correct.

arXiv preprint arXiv:2211.00053 (2022).

[49] Zhenyu Wu, Qingkai Zeng, Zhihan Zhang, Zhaoxuan Tan, Chao Shen, and Meng Jiang. 2024. Large language models can self-correct with key condition verification. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 12846–12867.

[50] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 12–22.

[51] Zhiheng Xi, Dingwen Yang, Jixuan Huang, Jiafu Tang, Guanyu Li, Yiwen Ding, Wei He, Boyang Hong, Shihan Do, Wenyu Zhan, et al. 2024. Enhancing LLM Reasoning via Critique Models with Test-Time and Training-Time Supervision. *arXiv preprint arXiv:2411.16579* (2024).

[52] Yu Xia, Rui Zhong, Hao Gu, Wei Yang, Chi Lu, Peng Jiang, and Kun Gai. 2025. Hierarchical Tree Search-based User Lifelong Behavior Modeling on Large Language Model. *arXiv preprint arXiv:2505.19505* (2025).

[53] Lanling Xu, Junjie Zhang, Bingqian Li, Jinpeng Wang, Mingchen Cai, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Prompting large language models for recommender systems: A comprehensive framework and empirical analysis. *arXiv preprint arXiv:2401.04997* (2024).

[54] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[55] Wenhao Yu, Zhihan Zhang, Zhenwen Liang, Meng Jiang, and Ashish Sabharwal. 2023. Improving language models via plug-and-play retrieval feedback. *arXiv preprint arXiv:2305.14002* (2023).

[56] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems* (2023).

[57] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).

[58] Xin Zheng, Jie Lou, Boxi Cao, Xueru Wen, Yuqiu Ji, Hongyu Lin, Yaojie Lu, Xianpei Han, Debing Zhang, and Le Sun. 2024. Critic-cot: Boosting the reasoning abilities of large language model via chain-of-thoughts critic. *arXiv preprint arXiv:2408.16326* (2024).

[59] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.